

✓ 반도체장비예지보전실무_01_연습문제02 유방암 진단 예측_풀이

```
# 필요한 라이브러리 임포트
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

pd.set_option('display.float_format', lambda x: '%.4f' % x)
```

✓ 단계 1: 데이터 로드 및 탐색

```
# -----
# 단계 1: 데이터 로드 및 탐색
# -----
# load_breast_cancer 데이터셋 로드
cancer = load_breast_cancer(as_frame=True)

cancer.keys()

dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
```

```
# 특성(X)과 타겟(y) 분리
X = cancer.data
y = cancer.target

# DataFrame으로 변환하여 데이터 탐색 용이하게 만들기
# df = pd.DataFrame(X, columns=cancer.feature_names)
# df['target'] = y

print("===== 데이터셋 상위 5개 행 확인 =====")
print(X.head())
print("\n===== 데이터셋 정보 확인 =====")
X.info()
print("\n===== 타겟 클래스 분포 확인 =====")
# 0: 악성(Malignant), 1: 양성(Benign)
print(y.value_counts(normalize=True))
# 이 데이터셋에는 결측치가 없으므로 추가 처리가 필요 없습니다.
```

숨겨진 출력 표시

```
X.describe()
```

숨겨진 출력 표시

✓ 단계 2: 데이터 전처리 (특성 스케일링)

```
# -----
# 단계 2: 데이터 전처리 (특성 스케일링)
# -----
# 로지스틱 회귀는 경사 하강법 기반이므로, 특성 스케일링이 필수적입니다.
# StandardScaler를 사용하여 평균 0, 표준편차 1로 스케일 조정
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)
X_scaled_df.describe()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...
count	569.0000	569.0000	569.0000	569.0000	569.0000	569.0000	569.0000	569.0000	569.0000	569.0000	..
mean	-0.0000	0.0000	-0.0000	-0.0000	-0.0000	0.0000	0.0000	-0.0000	0.0000	0.0000	..
std	1.0009	1.0009	1.0009	1.0009	1.0009	1.0009	1.0009	1.0009	1.0009	1.0009	..
min	-2.0296	-2.2292	-1.9845	-1.4544	-3.1121	-1.6101	-1.1149	-1.2618	-2.7441	-1.8199	..
25%	-0.6894	-0.7260	-0.6920	-0.6672	-0.7110	-0.7471	-0.7437	-0.7379	-0.7032	-0.7226	..
50%	-0.2151	-0.1046	-0.2360	-0.2952	-0.0349	-0.2219	-0.3422	-0.3977	-0.0716	-0.1783	..
75%	0.4694	0.5842	0.4997	0.3635	0.6362	0.4939	0.5261	0.6469	0.5308	0.4710	..
max	3.9713	4.6519	3.9761	5.2505	4.7709	4.5684	4.2436	3.9279	4.4848	4.9109	..

8 rows × 30 columns

✓ 단계 3: 훈련 세트와 테스트 세트 분할

```
# -----
# 단계 3: 훈련 세트와 테스트 세트 분할
# -----
# stratify=y 옵션을 사용하여 타겟 클래스 비율을 훈련/테스트 세트에 동일하게 분배
X_train, X_test, y_train, y_test = \
    train_test_split(X_scaled, y, test_size=0.2,
                    random_state=42, stratify=y)

print(f'\n훈련 세트 크기: {X_train.shape}')
print(f'\ntest 세트 크기: {X_test.shape}')
print(f'\ntest 세트 타겟 분포: \n
{pd.Series(y_test).value_counts(normalize=True)}')

print(f'\ntest 세트 타겟 분포: \n'
      f'{pd.Series(y_test).value_counts(normalize=True)}')
```

```
훈련 세트 크기: (455, 30)
테스트 세트 크기: (114, 30)
테스트 세트 타겟 분포:
target
1    0.6316
0    0.3684
Name: proportion, dtype: float64
```

✓ 단계 4: 로지스틱 회귀 모델 훈련

```
# -----
# 단계 4: 로지스틱 회귀 모델 훈련
# -----
# LogisticRegression 모델 객체 생성
model = LogisticRegression(random_state=42)

# fit() 메서드를 사용하여 모델 훈련
# 이 과정에서 교차 엔트로피 손실을 최소화하는 최적의 W와 b를 찾습니다.
model.fit(X_train, y_train)

# 학습된 가중치(W)와 편향(b) 확인
print("\n===== 학습된 모델 가중치와 편향 =====")
print("가중치 (Coefficients):")
for name, coef in zip(cancer.feature_names, model.coef_[0]):
    print(f' - {name}: {coef:.4f}')
print(f'\n편향 (Intercept): {model.intercept_[0]:.4f}')
```

```
# 가중치의 부호는 해당 특성이 악성(0) 또는 양성(1) 분류에
# 미치는 영향의 방향을 의미합니다.
```

```
===== 학습된 모델 가중치와 편향 =====
```

```
가중치 (Coefficients):
```

```
- mean radius: -0.5312
- mean texture: -0.5486
- mean perimeter: -0.4945
- mean area: -0.5486
- mean smoothness: -0.2381
- mean compactness: 0.6550
- mean concavity: -0.6086
- mean concave points: -0.7136
- mean symmetry: -0.1581
- mean fractal dimension: 0.2000
- radius error: -1.1233
- texture error: 0.2396
- perimeter error: -0.5922
- area error: -0.8990
- smoothness error: -0.1688
- compactness error: 0.6272
- concavity error: 0.1716
- concave points error: -0.4506
- symmetry error: 0.3563
- fractal dimension error: 0.4342
- worst radius: -0.9563
- worst texture: -1.2377
- worst perimeter: -0.7680
- worst area: -0.9360
- worst smoothness: -0.7200
- worst compactness: 0.0740
- worst concavity: -0.8234
- worst concave points: -0.9426
- worst symmetry: -0.9252
- worst fractal dimension: -0.1945
```

```
편향 (Intercept): 0.1659
```

✓ 단계 5: 모델 예측 및 평가

```
# -----
# 단계 5: 모델 예측 및 평가
# -----
# 테스트 세트 예측
y_pred = model.predict(X_test) # 1, 0
y_pred_proba = model.predict_proba(X_test) # 확률 예측

# 모델 평가 지표 계산
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print("\n===== 모델 평가 (테스트 세트) =====")
print(f'정확도 (Accuracy): {accuracy:.4f}')
print(f'정밀도 (Precision): {precision:.4f}')
print(f'재현율 (Recall): {recall:.4f}')
print(f'F1 Score: {f1:.4f}')
print(f'\n혼동 행렬 (Confusion Matrix):\n{conf_matrix}')

# TP, TN, FP, FN 값 파악
# [[TN, FP],
#  [FN, TP]]
tn, fp, fn, tp = conf_matrix.ravel()
print(f'\nTP (진양성): {tp}, TN (진음성): {tn},
      f' FP (위양성): {fp}, FN (위음성): {fn}''')
```

```
===== 모델 평가 (테스트 세트) =====
정확도 (Accuracy): 0.9825
```

정밀도 (Precision): 0.9861
재현율 (Recall): 0.9861
F1 Score: 0.9861

혼동 행렬 (Confusion Matrix):
[[41 1]
 [1 71]]

TP (진양성): 71, TN (진음성): 41, FP (위양성): 1, FN (위음성): 1

✓ 단계 6: 결과 해석 및 시사점 도출

```
# -----  
# 단계 6: 결과 해석 및 시사점 도출  
# -----  
print("\n==== 결과 해석 및 시사점 =====")  
# 가중치의 절댓값이 클수록 분류에 중요한 역할을 합니다.  
coef_df = pd.DataFrame(model.coef_.T,  
                        index=cancer.feature_names, columns=['Coefficient'])  
coef_df['Abs_Coefficient'] = coef_df['Coefficient'].abs()  
sorted_coef = coef_df.sort_values(by='Abs_Coefficient', ascending=False)  
print("악성/양성 분류에 가장 큰 영향을 미치는 특성 (상위 5개):")  
print(sorted_coef.head())  
  
# 의료 분야에서는 FN(실제 악성인데 양성으로 오진) 오류가  
# FP(실제 양성인데 악성으로 오진)보다 훨씬 치명적입니다.  
# FN은 치료 시기를 놓쳐 생명을 위협할 수 있기 때문입니다.  
# 이 모델의 재현율(Recall)을 높이는 것이 중요한 목표가 될 수 있습니다.  
# 재현율은 실제 악성 환자 중 얼마나 많은 수를  
# 올바르게 악성으로 예측했는지를 나타냅니다.
```

```
==== 결과 해석 및 시사점 =====  
악성/양성 분류에 가장 큰 영향을 미치는 특성 (상위 5개):  
      Coefficient  Abs_Coefficient  
worst texture      -1.2377         1.2377  
radius error       -1.1233         1.1233  
worst radius       -0.9563         0.9563  
worst concave points -0.9426         0.9426  
worst area         -0.9360         0.9360
```

› 전체 코드

↳ 숨겨진 셀 1개